

[+](#) DEVOPS TOPICS

What is DevOps?

Use free DevOps template

Try Compass for free

Improve your developer experience, catalog all services, and increase software health.

Try it free

DevOps is a set of [practices](#), [tools](#), and a [cultural philosophy](#) that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.

The DevOps movement [began around 2007](#) when the software development and IT operations communities raised concerns about the traditional software development model, where developers who wrote code worked apart from operations who deployed and supported the code. The term DevOps, a combination of the words development and operations, reflects the process of integrating these disciplines into one, continuous



HOW DOES DEVOPS WORK:

A DevOps team includes developers and IT operations working collaboratively throughout the product lifecycle, in order to increase the speed and quality of software deployment. It's a new way of working, a cultural shift, that has significant implications for teams and the organizations they work for.

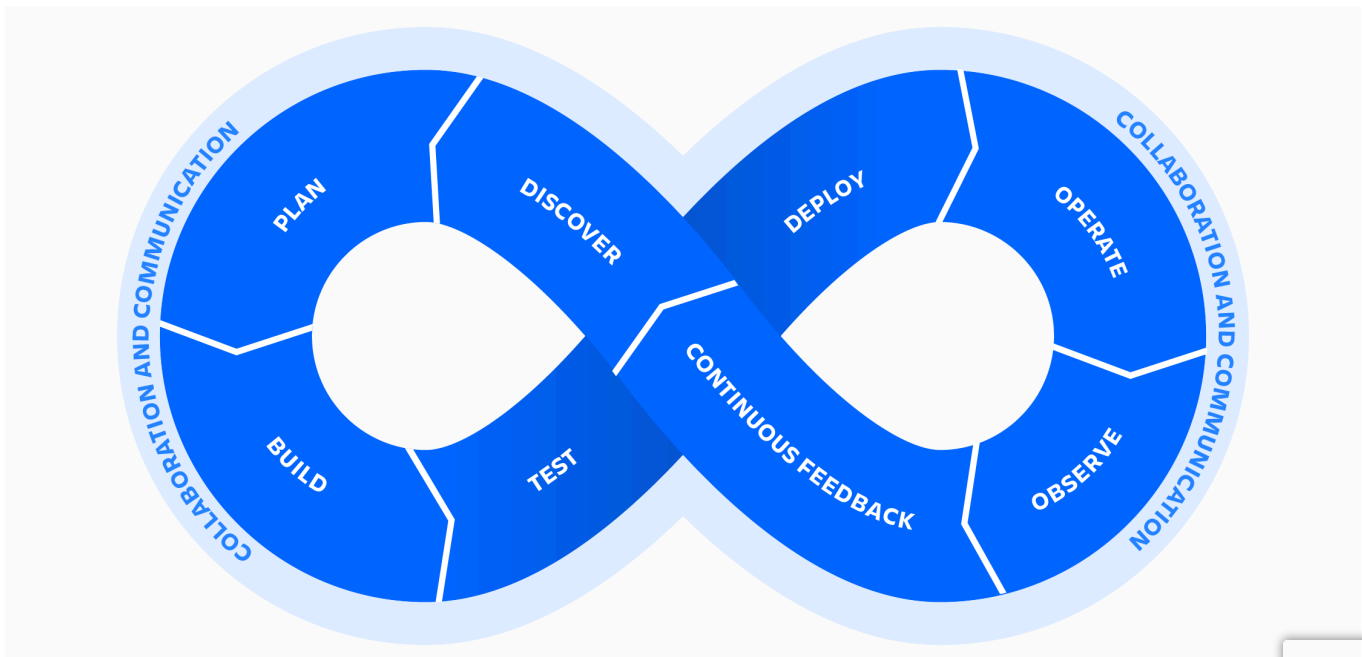
Under a DevOps model, development and operations teams are no longer “siloeD.” Sometimes, these two teams merge into a single team where the engineers work across the entire application lifecycle – from development and test to deployment and operations – and have a range of multidisciplinary skills.

DevOps teams use tools to automate and accelerate processes, which helps to increase reliability. A DevOps toolchain helps teams tackle important DevOps fundamentals including continuous integration, continuous delivery, automation, and collaboration.

DevOps values are sometimes applied to teams other than development. When security teams adopt a DevOps approach, security is an active and integrated part of the development process. This is called [DevSecOps](#).

The DevOps lifecycle

Because of the continuous nature of DevOps, practitioners use the infinity loop to show how the phases of the DevOps lifecycle relate to each other. Despite appearing to flow sequentially, the loop symbolizes the need for constant collaboration and iterative improvement throughout the entire lifecycle.



The DevOps lifecycle consists of eight phases representing the processes, capabilities, and tools needed for development (on the left side of the loop) and operations (on the right side of the loop). Throughout each phase, teams collaborate and communicate to maintain alignment, velocity, and quality.

[Learn more about communication and collaboration →](#)

Discover

Building software is a team sport. In preparation for the upcoming sprint, teams must workshop to explore, organize, and prioritize ideas. Ideas must align to strategic goals and deliver customer impact. Agile can help guide DevOps teams.

[Learn more about product discovery →](#)

Plan

DevOps teams should adopt agile practices to improve speed and quality. Agile is an iterative approach to project management and software development that helps teams break work into smaller pieces to deliver incremental value.

[Learn more about agile →](#)

Build

Git is a free and open source version control system. It offers excellent support for branching, merging, and rewriting repository history, which has led to many innovative and powerful workflows and tools for the development build process.

[Learn more about git →](#)

Test

Continuous integration (CI) allows multiple developers to contribute to a single shared repository. When code changes are merged, automated tests are run to ensure correctness before integration. Merging and testing code often help development teams gain reassurance in the quality and predictability of code once deployed.

[Learn more about CI →](#)

Deploy

Continuous deployment (CD) allows teams to release features frequently into production in an automated fashion. Teams also have the option to deploy with feature flags, delivering new code to users steadily and methodically rather than all at once. This approach improves velocity, productivity, and sustainability of software development teams.

[Learn more about CD →](#)

Operate

Manage the end-to-end delivery of IT services to customers. This includes the practices involved in design, implementation, configuration, deployment, and maintenance of all IT infrastructure that supports an organization's services.

[Learn more about operations →](#)

Observe

Quickly identify and resolve issues that impact product uptime, speed, and functionality. Automatically notify your team of changes, high-risk actions, or failures, so you can keep services on.

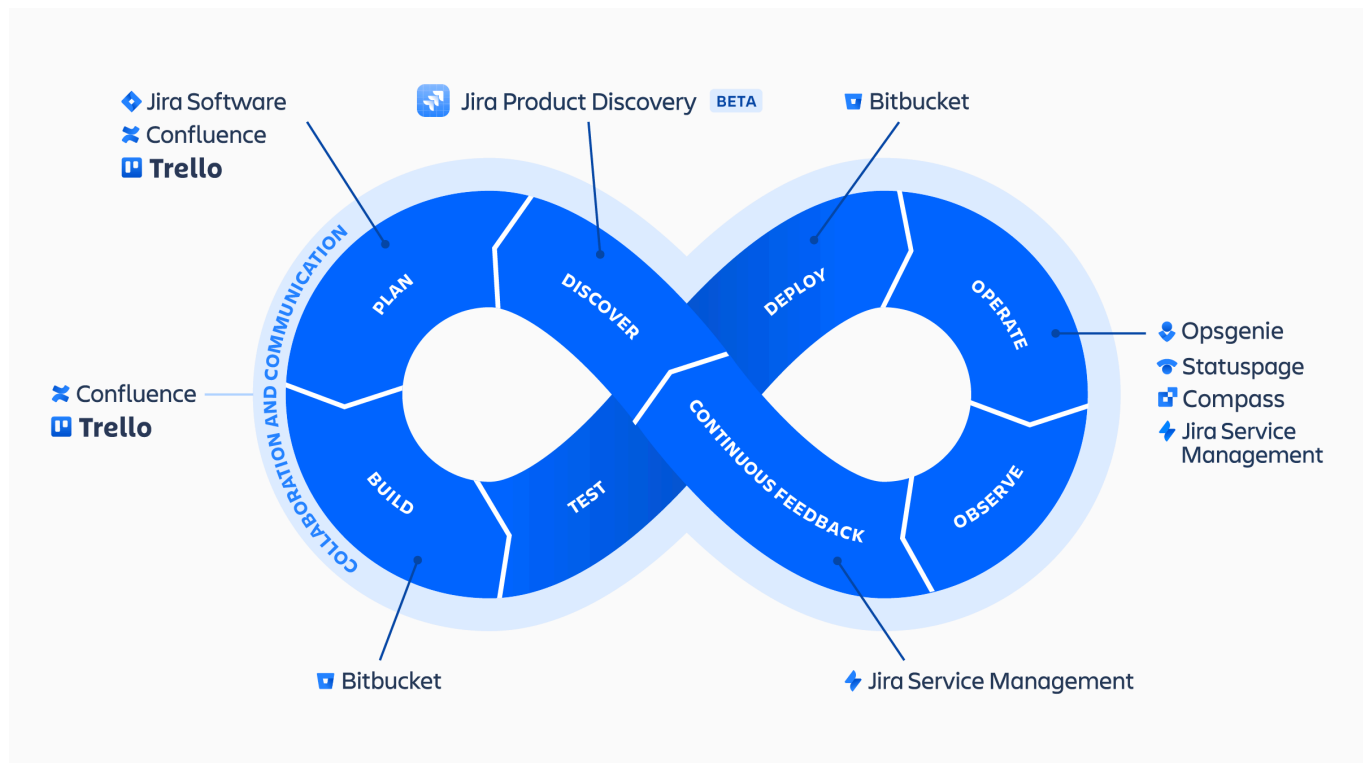
[Learn more about monitoring →](#)

Continuous feedback

DevOps teams should evaluate each release and generate reports to improve future releases. By gathering continuous feedback, teams can improve their processes and incorporate customer feedback to improve the next release.

[Learn more about continuous feedback →](#)

DevOps tools



[DevOps tools](#) address the key phases of the DevOps lifecycle. They empower DevOps practices by helping to improve collaboration, reduce context-switching, introduce automation, and enable observability and monitoring.

DevOps toolchains usually follow two approaches: an all-in-one or open toolchain. An all-in-one toolchain offers a complete solution that usually doesn't integrate with other third-party tools, while an open toolchain allows for customization with different tools. There are [pros and cons to both approaches](#).

An example of an open DevOps toolchain is Atlassian's Open DevOps solution, which includes Jira as a foundation and integrates with leading vendors and marketplace apps.

[Read more about DevOps tools →](#)

What are the benefits of DevOps?

In Atlassian's [2020 DevOps Trends survey](#), 99 percent of respondents said that DevOps had a positive impact on their organization. The [benefits of DevOps](#) include faster and easier releases, team efficiency, increased security, higher quality products, and consequently happier teams and customers.



Speed

Teams that practice DevOps release deliverables more frequently, with higher quality and stability. In fact, the DORA [2019 State of DevOps](#) report found that elite teams deploy 208 times more frequently and 106 times faster than low-performing teams. Continuous delivery allows teams to build, test, and deliver software with automated tools.



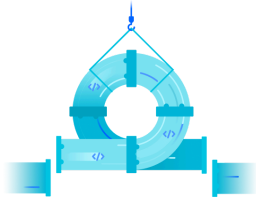
Improved Collaboration

The foundation of DevOps is a culture of collaboration between developers and operations teams, who share responsibilities and combine work. This makes teams more efficient and saves time related to work handoffs and creating code that is designed for the environment where it runs.



Rapid deployment

By increasing the frequency and velocity of releases, DevOps teams improve products rapidly. A competitive advantage can be gained by quickly releasing new features and repairing bugs.



Quality and reliability

Practices like continuous integration and continuous delivery ensure changes are functional and safe, which improves the quality of a software product. Monitoring helps teams keep informed of performance in real-time.



Security

By integrating security into a continuous integration, continuous delivery, and continuous deployment pipeline, [DevSecOps](#) is an active, integrated part of the development process. Security is built into the product by integrating active security audits and security testing into agile development and DevOps workflows.

What are the challenges of adopting DevOps?

Habits are hard to break. Teams entrenched in siloed ways of working can struggle with, or even be resistant to, overhauling team structures to embrace DevOps practices. Some teams may mistakenly believe new tools are sufficient to adopt DevOps. Yet, DevOps is a combination of people, tools, and culture. Everyone on a DevOps team must understand the entire value stream – from ideation, to development, to the end user experience. It requires breaking down silos in order to collaborate throughout the product lifecycle.

"DevOps isn't any single person's job. It's everyone's job."

Robert Krohn

HEAD OF ENGINEERING, DEVOPS AT ATlassian

Moving from a legacy infrastructure to using [Infrastructure as Code](#) (IaC) and [microservices](#) can offer faster development and innovation, but the increased operational workload can be challenging. It's best to build out a strong foundation of automation, configuration management, and continuous delivery practices to help ease the load.

An over-reliance on tools can detract teams from the necessary foundations of DevOps: the team and organization structure. Once a structure is established, the processes and team should come next and the tools should follow.

How to adopt DevOps

Adopting DevOps first requires a commitment to evaluating and possibly changing or removing any teams, tools, or processes your organization currently uses. It means building the necessary infrastructure to give teams the autonomy to build, deploy, and manage their products without having to rely too heavily on external teams.

DevOps culture

A [DevOps culture](#) is where teams embrace new ways of working that involve greater collaboration and communication. It's an alignment of people, processes, and tools toward a more unified customer focus. Multidisciplinary teams take accountability for the entire lifecycle of a product.

Continuous learning

Organizations that do DevOps well are places where experimentation and some amount of risk-taking are encouraged. Where thinking outside the box is the norm, and failure is understood to be a natural part of learning and improving.

Agile

[Agile methodologies](#) are immensely popular in the software industry since they empower teams to be inherently flexible, well-organized, and capable of responding to change. DevOps is a cultural shift that fosters collaboration between those who build and

maintain software. When used together, agile and DevOps result in high efficiency and reliability.

DevOps practices

Continuous integration

[Continuous integration](#) is the practice of automating the integration of code changes into a software project. It allows developers to frequently merge code changes into a central repository where builds and tests are executed. This helps DevOps teams address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

[Learn about continuous integration →](#)

Continuous delivery

Continuous delivery expands upon continuous integration by automatically deploying code changes to a testing/production environment. It follows a [continuous delivery pipeline](#), where automated builds, tests, and deployments are orchestrated as one release workflow.

[Learn about continuous delivery →](#)

Situational awareness

It is vital for every member of the organization to have access to the data they need to do their job as effectively and quickly as possible. Team members need to be alerted of failures in the deployment pipeline – whether systemic or due to failed tests – and receive timely updates on the health and performance of applications running in production. Metrics, logs, traces, monitoring, and alerts are all essential sources of feedback teams need to inform their work.

[Learn about DevOps metrics →](#)

Automation

Automation is one of the most important DevOps practices because it enables teams to move much more quickly through the process of developing and deploying high-quality

software. With automation the simple act of pushing code changes to a source code repository can trigger a build, test, and deployment process that significantly reduces the time these steps take.

[Learn about DevOps automation best practices →](#)

Infrastructure as Code

Whether your organization has an on-premise data center or is completely in the cloud, having the ability to quickly and consistently provision, configure, and manage infrastructure is key to successful DevOps adoption. [Infrastructure as Code](#) (IaC) goes beyond simply scripting infrastructure configuration to treating your infrastructure definitions as actual code: using source control, code reviews, tests, etc.

[Learn about Infrastructure as Code →](#)

Microservices

[Microservices](#) is an architectural technique where an application is built as a collection of smaller services that can be deployed and operated independently from each other. Each service has its own processes and communicates with other services through an interface. This separation of concerns and decoupled independent function allows for DevOps practices like continuous delivery and continuous integration.

[Learn about microservices →](#)

Monitoring

DevOps teams monitor the entire development lifecycle – from planning, development, integration and testing, deployment, and operations. This allows teams to respond to any degradation in the customer experience, quickly and automatically. More importantly, it allows teams to “shift left” to earlier stages in development and minimize broken production changes.

[Learn about DevOps monitoring →](#)

Get started with DevOps



The easiest way to get started with DevOps is to identify a small value stream (for example a small supporting app or service) and start experimenting with some DevOps practices. As with software development, it is far easier to transform a single stream with a small group of stakeholders than to attempt an all-at-once organizational transition to a new way of working.

Atlassian offers an [Open DevOps solution](#) that provides end-to-end DevOps processes with Atlassian and other third-party tools. With Jira as the backbone, teams can use Atlassian products or bring their favorite products to the open toolchain. The Atlassian ecosystem offers a robust array of integrations and add-ons, allowing teams to customize their toolchain to meet their needs.

Be sure to check out our DevOps tutorials for [automation](#), [testing](#), [security](#), [observability](#), [feature flagging](#), and [continuous delivery](#).



[DevOps automation tutorials →](#)

[DevOps testing tutorials →](#)

[DevOps security tutorials →](#)

[DevOps observability tutorials →](#)

[DevOps feature flagging tutorials →](#)

[DevOps continuous delivery tutorials →](#)

Recommended for you

DevOps Best Practices

[Learn more DevOps best practices →](#)

DevOps Blog

[See what's new →](#)

Find answers and get connected

[Engage with the Community →](#)



Company

Careers

Events

Blogs

Investor Relations

Atlassian Foundation

Press kit

Contact us

PRODUCTS

Rovo

Jira

Jira Align

Jira Service Management

Confluence

Loom

Trello

Bitbucket

See all products →

RESOURCES

Technical support

Purchasing & licensing

Atlassian Community

Knowledge base

Marketplace

My account

Create support ticket →

LEARN

Partners



[Training & certification](#)

[Documentation](#)

[Developer resources](#)

[Enterprise services](#)

[See all resources →](#)

Copyright © 2026 Atlassian

[Privacy policy](#)

[Terms](#)

[Impressum](#)

 [English ▼](#)